Tutorial: Simulated RF Signal Generation with Matlab

Anagh Mishra, Boyang Wang

Data and System Security Lab (UCDaSec), Department of ECE, University of Cincinnati
Last modified: December 2025; Status: Completed

1 Introduction

What This Document is About. This document serves as a comprehensive guide for generating a customizable simulated I/Q dataset using MATLAB. The dataset consists of I/Q frames extracted from RF (Radio Frequency) signals, which are transmitted from a sender to a receiver with different modulations. The channels and other wireless components are simulated with MATLAB toolboxes. While the dataset is intended for training and testing modulation classification models, the data generation pipeline can also be utilized for generating simulated RF signals for other applications (e.g., radio/device fingerprinting)

This tutorial outlines the required setup, user-defined parameters, and step-by-step instructions for dataset generation. Additionally, it provides insight into the internal workings of the program to help users understand how the data is constructed and how various signal impairments are applied. Our source code, example datasets, and tutorial videos associated with this tutorial document can be found on our GitHub repository.[1]

Pre-Requisites. To follow the content of this document, the readers are expected to have a basic understanding of signals and systems and wireless communication. Furthermore, familiarity with MATLAB would allow readers to better follow the code presented in the document.

Special Note. There are different ways to generate RF data that simulate wireless communication. We present a way that works for us in this document and acknowledge that this may not be the most effective way to achieve the goal.

Acknowledgments. The research and education efforts associated with this document are supported by National Science Foundation (CNS-2225160, CNS-2225161).

[1] `https://github.com/UCdasec/RadioShift`

2 Software and Hardware Settings

We use the following hardware and software in our simulation pipeline.

- Hardware: A desktop with an Intel i9 CPU, 16 GB memory, and a NVIDIA 4080 GPU. At a minimum, we use a virtual machine image running Linux (Ubuntu 22.04) with 24 cores, 8 GB memory, and 300 GB storage.
- Software: A relatively recent version of MATLAB installed. Versions 2024a, 2024b, and 2025a are all supported, with this tutorial specifically developed and tested using MATLAB 2025a (Version 25.1).
- Required MATLAB Toolboxes:
  - Communications Toolbox
  - DSP HDL Toolbox
  - DSP System Toolbox
  - Deep Learning HDL Toolbox
  - Deep Learning Toolbox
  - Fixed-Point Designer
  - HDL Coder
  - MATLAB Coder
  - Satellite Communications Toolbox
  - Signal Processing Toolbox
  - Wireless HDL Toolbox

To check which toolboxes are installed on your system, type `ver` into the MATLAB command window. This will list all currently available toolboxes.

Note. All CEAS students at the University of Cincinnati have access to a licensed copy of MATLAB, which should be sufficient for this project. MATLAB is officially supported on Windows and several Linux distributions. However, not all Linux environments are compatible (e.g., Fedora is not fully supported). Please verify OS compatibility on the official MATLAB system requirements page to ensure your setup is appropriate.

Access to Our Code. To access our codebase, clone the our GitHub repository[2]. This repository includes all necessary components to generate the dataset:

- `generateSignals.m` – the main function for dataset generation
- `tools.py` – a modified version of `tools.py` from our RadioFINN repository[3]
- `ModClassificationShiftSignalGeneration.mlx` – a live script providing a walkthrough of the generation process
- Helper files prefixed with `dlhdlhelper` – utility functions used internally

[2] `https://github.com/UCdasec/RadioShift`
[3] `https://github.com/UCdasec/RadioFINN`

– A `.wav` file – used for analog signal modulation
– Additional support files for signal configuration and model interfacing

Make sure to keep all files in the same working directory or add paths accordingly in MATLAB. Occasionally, MATLAB may encounter conflicting path issues, especially if it prioritizes functions from its built-in Examples library over the ones in your working directory. To check for potential conflicts, run `which <function_name> -all` in the command line. If the output indicates that a conflicting function is being loaded from a MATLAB Examples directory (e.g., `matlab/examples/...`), you should override it by adding your current workspace to the top of the search path by running `addpath(</worspace_path>)`

Additionally, when executing the `generateSignals.m` script to generate data, ensure you are doing so from within the src directory to avoid path resolution issues.

3 Overview of Our Pipeline

3.1 Description of Parameters

The `generateSignals.m` function is the primary tool used to generate a synthetic I/Q dataset in MATLAB. It offers a set of configurable parameters that allow users to control the properties and structure of the generated dataset. Below, we provide descriptions of each parameter, along with an example demonstrating how to use the function from the MATLAB Command Line.

Table 1: Description of required parameters

Parameter	**Type**	**Description**
`frames_per_mod_type`	Integer	Specifies the number of frames to generate per modulation + SNR combination.
`MAX_PPM`	Integer	Sets the maximum sampling frequency offset in parts per million (PPM). *Note: A value of 0 means no offset. A realistic upper limit is typically around 20.*
`Path`	String	Specifies the directory path where the dataset will be saved.
`FolderName`	String	Specifies the folder name in which the dataset will be saved.

Table 2: Description of optional parameters

Parameter	**Type**	**Description**
`spf`	Integer	Number of I/Q samples per frame. *Default: 1024*
`sps`	Integer	Number of samples per symbol. *Default: 8*
`hard_set_offset`	Boolean (1/0)	If true, applies a fixed offset equal to `MAX_PPM`. If false, applies a random offset in the range [0, `MAX_PPM`]. *Default: true*
`freq_scale_factor`	Integer (-1/1)	Applies a positive or negative sampling frequency offset. Set to 1 to increase or -1 to decrease the sampling rate at the receiver. *Default: 1*
`channelType`	Boolean(1/0)	If true, applies a Rayleigh channel. If false, apples Rician channel. *Default: 0*

3.2 Signal Characteristics

The purpose of the `generateSignals.m` function is to create simulated signals representing a variety of modulation schemes. These modulation schemes and channel conditions with impairments

Table 3: Example values for parameters used in our example dataset generation

Parameter	Value
`frames_per_mod_type`	4096
`MAX_PPM`	0
`Path`	No Specified Value
`FolderName`	No Specified Value
`spf`	1024 (*Default Value*)
`sps`	8 (*Default Value*)
`hard_set_offset`	1 or True (*Default Value*)
`freq_scale_factor`	1 (*Default Value*)

are implemented using built-in MATLAB libraries and customizable channel modeling modules. The function aims to replicate the structure of the RadioML dataset[4]

Specifically, our pipeline follows a similar format as the RadioML dataset but supports a smaller number of modulation schemes (15 rather than 27). On the other hand, our pipeline stores I/Q samples in both 8-bit integer and 32-bit float formats. As a result, each entry in the generated dataset includes the following key components:

- I/Q samples in 8-bit integer format
- I/Q samples in 32-bit float format
- Signal-to-noise ratio (SNR) value
- Modulation label index

Furthermore, since MATLAB supports and requires datasets larger than 2 GB to be stored in the `.h5` format, the generated dataset is saved accordingly—consistent with the format used in the RadioML 2021 dataset. Below is a list of characteristics of the dataset generated by this function

Table 4: Modulation schemes and signal parameters for dataset generation

Parameter	Value
Digital Modulations	BPSK, QPSK, 8PSK, 16/32/64/128/256 QAM, 16/32/64/128 APSK
Analog Modulations	FM, DSB-AM, SSB-AM
SNR Levels	0 dB to 30 dB (2 dB increments)
`sps` (Samples per symbol)	8
`spf` (Samples per frame)	1024
`fs` (Sampling rate)	200 kHz
Center Frequencies	902 MHz (Digital), 100 MHz (Analog)

along with parameters that can be leveraged by the user to customize the dataset.

[4] A well-known synthetic I/Q dataset for modulation classification: `https://www.deepsig.ai/datasets/`

– **Modulation Schemes (can be further extended if needed):**
  - Digital Modulations: BPSK, QPSK, 8PSK, 16/32/64/128/256 QAM, 16/32/64/128 APSK
  - Analog Modulations: FM, DSB-AM, SSB-AM
– **Signal Parameters (can be further extended if needed):**
  - SNR levels range from 0 dB to 30 dB with 2 dB increments
  - 8 samples per symbol (`sps`)
  - 1024 samples per frame (`spf`)
  - 200 kHz sampling rate (`fs`)
  - Two center frequencies: 902 MHz and 100 MHz
– **Dataset Specs:**
  - File Name: `MatGenData.m`
  - File Format: `.h5`
  - Key-Value Pairs:

Key	Value
`all_IQ_float32`	Array of IQ frames in 32-bit float format
`all_IQ_int8`	Array of IQ frames in 8-bit integer format
`all_SNRs`	Array of SNR values corresponding to each frame by index
`all_labels`	Modulation label index corresponding to each frame by index

3.3 Demonstration of Dataset Generation

Below is a step-by-step procedure for generating a dataset, assuming all prior setup steps have been completed and the values of parameters have been assigned.

Step 1: Launch MATLAB, either with or without the user interface (UI). For demonstration purposes, we run MATLAB without the UI to highlight how easily a dataset can be generated simply by running command lines.

```
# launch MATLAB in terminal
$ matlab -nodesktop -nodisplay
```

Step 2: Navigate to the src directory of the RadioShift repository. Use the `pwd` command to confirm that your current directory ends with `RadioShift/src` as shown in Fig. 2.

Step 3: Run the `generateSignal.m` function with the appropriate parameters. For this demonstration, `generateSignal` was run with 4096 frames per modulation type per SNR, 0 for `MAX_PPM`, and the resulting dataset was saved to `/home/anagh/Repos/RadioShift`.

As shown in Fig. 3, a total of 983,040 frames are generated (16 SNR levels $\times$ 15 modulation schemes $\times$ 4096 frames); labels of I/Q frames are assigned chronologically to each modulation typel

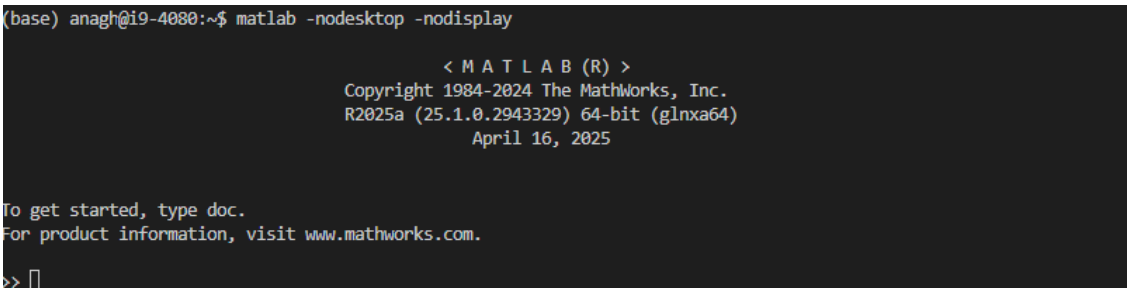

Fig. 1: Launching MATLAB in command line via terminal

dataset generation took approximately 10 minutes; the dataset is stored in `.mat` format, with the file named `MatGenData.mat`. The final generated dataset, consisting of 8-bit integer samples and 32-bit float samples, is approximately 10 GB in size.

Fig. 2: Changing to ./RadioShift/src Directory

```
>> generateSignals(4096, 0, "/home/anagh/Documents/Datasets", "0_PPM_OFFSET")
Data file directory is /home/anagh/Documents/Datasets/0_PPM_OFFSET
/home/anagh/Documents/Datasets/0_PPM_OFFSET exists and generated signals will be saved here
Hardset PPM Offset: 1
Maximum Clock Offset (PPM): 0
A total of 983040 frames will be generated...
00:00:00 - Generating BPSK frames
    "Modulation: "    "BPSK"    " → Label Index: "    "0"

00:00:31 - Generating QPSK frames
    "Modulation: "    "QPSK"    " → Label Index: "    "1"

00:01:03 - Generating 8PSK frames
    "Modulation: "    "8PSK"    " → Label Index: "    "2"

00:01:35 - Generating 16QAM frames
    "Modulation: "    "16QAM"    " → Label Index: "    "3"

00:02:09 - Generating 32QAM frames
    "Modulation: "    "32QAM"    " → Label Index: "    "4"

00:02:44 - Generating 64QAM frames
    "Modulation: "    "64QAM"    " → Label Index: "    "5"

00:03:18 - Generating 128QAM frames
    "Modulation: "    "128QAM"    " → Label Index: "    "6"

00:03:53 - Generating 256QAM frames
    "Modulation: "    "256QAM"    " → Label Index: "    "7"

00:04:28 - Generating 16APSK frames
    "Modulation: "    "16APSK"    " → Label Index: "    "8"

00:05:00 - Generating 32APSK frames
    "Modulation: "    "32APSK"    " → Label Index: "    "9"

00:05:33 - Generating 64APSK frames
    "Modulation: "    "64APSK"    " → Label Index: "    "10"

00:06:06 - Generating 128APSK frames
    "Modulation: "    "128APSK"    " → Label Index: "    "11"

00:06:39 - Generating FM frames
    "Modulation: "    "FM"    " → Label Index: "    "12"

00:07:21 - Generating AM-DSB-SC frames
    "Modulation: "    "AM-DSB-SC"    " → Label Index: "    "13"

00:07:57 - Generating AM-SSB-SC frames
    "Modulation: "    "AM-SSB-SC"    " → Label Index: "    "14"

Saving Int8 Dataset
Saving Float32 Dataset
```

Fig. 3: Console Output of Data Generation

3.4 Loading a Dataset using `tools.py`

Since the `radioml_21_dataset` class in `tools.py` is already designed to preprocess the RadioML dataset (`RADIOML_2021_07_INT8.hdf5` file) and extract labels, it was deemed more appropriate to modify this existing class rather than create a new one specifically for MATLAB-generated data.

Specifically, to keep the loader compatible with both the RadioML dataset and our generated MATLAB datasets, the constructor now consists of six parameters that make the preprocessing stage fully configurable. The `dataset_path` parameter tells the loader where to find the `.h5` file. The boolean `matgen_data` signals whether MATLAB generated datasets are going to be used or the RadioML Dataset is going to be used. The `frames_per_mod` parameter expresses the expected number of frames per modulation–SNR pair, while `snr_lb` and `snr_ub` bound the SNR values to load—useful because RadioML spans -20 dB to +30 dB, whereas our MATLAB data covers 0 dB to +30 dB. Lastly, `fp_data` flags whether the MATLAB 32-bit-float I/Q samples instead of 8-bit integers. Together, these arguments let a single class flexibly handle multiple sources, SNR windows, and sample formats without further structural changes.

Table 5: Parameters for loading and configuring the dataset

Parameter	**Type / Default**	**Purpose**
`dataset_path`	`str` (required)	Full path to the `.h5` dataset file.
`matgen_data`	`bool` (required)	`True` : Using MATLAB-generated data. `False`: Using RadioML 2021 data.
`frames_per_mod`	`int`, default 4096	Frames expected per modulation scheme at each SNR value.
`snr_lb`	`float`, default 0.0	Lower bound of the SNR range (in dB) to load.
`snr_ub`	`float`, default 30.0	Upper bound of the SNR range (in dB) to load.
`fp_data`	`bool`, default `False`	`True` : Loading 32-bit Float MATLAB dataset. `False`: Loading 8-bit Integer MATLAB dataset.

Using Modified tools.py. Because `tools.py` (our modified version) is now fully configurable, every location in the code that instantiates a `radioml_21_dataset` object must be updated. Each call should explicitly pass `dataset_path`, `matgen_data`, `fp_data`, and (where relevant) `snr_lb` so that the loader can distinguish among the three supported datasets—RadioML 2021 (INT8), MATLAB INT8, and MATLAB Float32.

The following code example highlights one such call, showing how the new arguments are supplied to ensure the correct preprocessing branch is selected.

```
# load data and create loaders
$ dataset = random_21_dataset(dataset, MatGenData, fp_data = fp_data, snr_lb =
    snr_lb)
```

To eliminate the need for hard-coding constructor arguments, two new parameters have been added to the `train_and_test` and `run_test` commands in `vanilla_model.py`. Alongside the existing `weights` parameter and `batch_size`, which is specific to `run_test`—the commands now accept the boolean parameters `matgen_data` and `fp_data`. These parameters correspond directly to those used in the dataset class: `matgen_data` selects the MATLAB-generated dataset, while `fp_data` indicates that the dataset contains 32-bit-float I/Q samples instead of 8-bit integers. By providing these parameters, the design introduces an abstraction layer, so model trainers do not need to specify exact SNR ranges or frames per modulation/SNR — this is all managed internally by the program.

Note. Something that does need to be hard-coded is the path where datasets are located. As shown in the following code example, in both the `train_and_test` and the `run_test` command, there exists a conditional where the dataset is selected based on the parameters passed in during run-time.

```
if(MatGenData):
    dataset_path = Path(mat_path)
    snr_lb = 0
else:
    dataset_path = Path(radioml_path)
    snr_lb = -20
```

The path values in the conditional cases need to be hard-coded since it is expected that the path location of the datasets will not move. The initialization of variables `mat_path` and `radioml_path` are at the top of `vanilla_model.py` file. An example is listed below.

```
# MATLAB Generated Dataset 0 PPM
mat_path = "/home/anagh/Documents/Datasets/0_PPM_OFFSET/MatGenData.h5"

# RadioML Dataset
radioml_path = "home/anagh/Documents/RADIOML_2012_07_INT8/RADIOML_2021_07_INT8.
    hdf5"
```

3.5 Running Train-and-Test using each dataset

Once a dataset is loaded, we can train a neural network and test it with `vanilla_model.py`. To do this, simply run the commands shown below, specifying the desired save location for the model weights via `desired_weights_path`. Specifically, we can run the following command to train and test a neural network with RadioML 2021 dataset

```
python vanilla_model.py train-and-test --base "<desired\_weights\_path>"
```

We can run the following command to train and test a neural network with our generated dataset (int8 format)

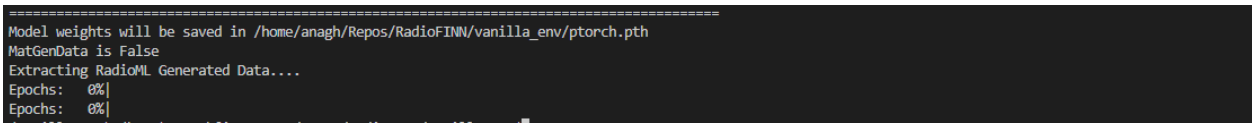

Fig. 4: Initial Output When Training with RadioML Dataset

```
python vanilla_model.py train-and-test --base "<desired\_weights\_path>" --
    matgendata
```

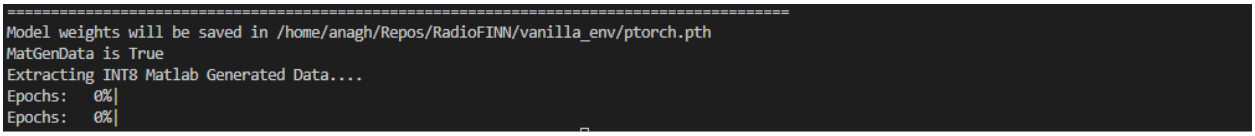

Fig. 5: Initial Output When Training with Our Generated Dataset (int8)

We can run the following command to train and test a neural network with our generated dataset (float32 format)

```
python vanilla_model.py train-and-test --base "<desired\_weights\_path>" --
    matgendata --fpdata
```

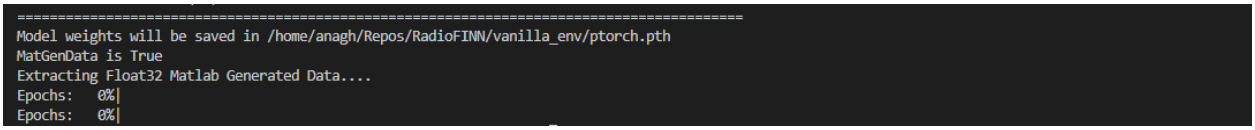

Fig. 6: Initial Output When Training with MATLAB Generated FLOAT32 Dataset

4 Conclusion

With the modifications and instructions detailed above, users are now equipped with all the necessary resources to generate datasets using MATLAB and seamlessly integrate them into the training and testing pipeline. The added flexibility in `tools.py` and `vanilla_model.py` ensures that both legacy and newly generated datasets can be handled efficiently, with minimal manual intervention. Whether using 8-bit integer or 32-bit float I/Q samples, the workflow now supports easy configuration, consistent preprocessing, and streamlined experimentation.